Homework Assignment: Taming the Imbalanced Dataset

Objective

In Module 1, we saw that while our models achieved high **Accuracy**, they often struggled with **Recall** (catching the actual bad credit/fraud cases). This is due to the class imbalance—there are far fewer "Risk" examples than "Safe" ones.

Your mission is to improve the **Recall** and **F1-Score** for the "Risk" class (Class 1) using the **Random Forest** model from the module. You will compare four distinct strategies to handle this imbalance.

The Dataset

Use the same German Credit Risk dataset from Module 1.

- **Target:** `0` (Safe) vs `1` (Risk).
- **Base Model:** `RandomForestClassifier(n_estimators=100, random_state=42)`.

Techniques to Implement & Compare

Compare the following four approaches against your baseline (Standard Random Forest):

1. Cost-Sensitive Learning (Class Weights)

Most Scikit-Learn classifiers (Logistic Regression, Random Forest, SVM) have a `class_weight` parameter.

- **Task:** Initialize your Random Forest with `class_weight='balanced'`.
- **Theory:** This tells the model to pay more "attention" to the minority class. It penalizes mistakes on the "Risk" class more heavily than mistakes on the "Safe" class.

2. Threshold Moving (Post-Processing)

By default, models predict Class 1 if the probability is > 0.5.

- **Task:** Instead of using `.predict()`, use `.predict_proba()`. Classify an example as "Risk" (1) if the probability is > 0.3 (or try 0.2, 0.4).
- **Theory:** Lowering the threshold increases Recall (catches more fraud) but lowers Precision (more false alarms). Find the "sweet spot" that maximizes F1.

3. Random Oversampling

- **Task:** Manually duplicate examples from the minority class (Risk) in your **Training Set** until it is roughly the same size as the majority class.
- **Library Hint:** You can use `sklearn.utils.resample` or the `imblearn` library if available.
- **Warning:** *Never* oversample your Test Set. Only the Training Set.

4. SMOTE (Synthetic Minority Over-sampling Technique) (Optional / Advanced)

- **Task:** Instead of just duplicating existing rows (which causes overfitting), generate *new* synthetic examples that are mathematically similar to the existing minority cases.

- **Library Hint:** Requires `from imblearn.over_sampling import SMOTE`.
- **Theory:** This creates clusters of minority class data, helping the model generalize better.

Deliverable: The Comparison Table

Produce a table summarizing your findings. It should look something like this:

Technique	Accuracy	Precision (Class 1)	Recall (Class 1)	F1-Score (Class 1)
Baseline (No Changes)	0.76	0.60	0.45	0.51
Class Weights	?	?	?	?
Threshold Moving (>0.3)	?	?	?	?
Oversampling	?	?	?	?

Reflection Questions

1. Which technique gave the highest **Recall**? Did it destroy your **Precision**?
2. Why is it dangerous to oversample data *before* splitting into Train and Test?
3. If you were the bank manager, would you prefer the model with high Precision or high Recall? (Consider the cost of a bad loan vs. the cost of an angry customer).